

DATA STRUCTURE

LAB PROGRAMS

13. a) Queue Insertion (Enqueue) Using Linked list

```
#include <stdio.h>

#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *front = NULL, *rear = NULL, *newNode;

    int n, i, value;

    printf("Enter number of elements to insert: ");

    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        newNode = (struct Node *)malloc(sizeof(struct Node));

        printf("Enter element: ");

        scanf("%d", &value);

        newNode->data = value;

        newNode->next = NULL;

        if(front == NULL)
        {
```

```
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
}
printf("Queue elements are: ");
struct Node *temp = front;
while(temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
return 0;
}
```

Output

```
Enter number of elements to insert: 5
Enter element: 10
Enter element: 20
Enter element: 30
Enter element: 40
Enter element: 50
Queue elements are: 10 20 30 40 50
```

```
=== Code Execution Successful ===
```

b) Queue Deletion (Dequeue) Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *front = NULL, *rear = NULL, *newNode, *temp;
    int n, i, value;
    printf("Enter number of elements in queue: ");
    scanf("%d", &n);
```

```

for(i = 0; i < n; i++)
{
    newNode = (struct Node *)malloc(sizeof(struct Node));
    scanf("%d", &value);
    newNode->data = value;
    newNode->next = NULL;
    if(front == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
}
if(front == NULL)
{
    printf("Queue Underflow");
}
else
{
    temp = front;
    printf("Deleted element = %d\n", front->data);
    front = front->next;
    free(temp);
}
printf("Queue after deletion: ");
temp = front;

```

```

while(temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
return 0;
}

```

Output

```

Enter number of elements in queue: 5
20 45 56 76 98
Deleted element = 20
Queue after deletion: 45 56 76 98

=== Code Execution Successful ===

```

c) Queue Display Using Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()

```

```

{
    struct Node *front = NULL, *rear = NULL, *newNode, *temp;
    int n, i, value;
    printf("Enter number of elements in queue: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        newNode = (struct Node *)malloc(sizeof(struct Node));
        scanf("%d", &value);
        newNode->data = value;
        newNode->next = NULL;
        if(front == NULL)
        {
            front = rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }
    }
    if(front == NULL)
    {
        printf("Queue is Empty");
    }
    else
    {
        printf("Queue elements are: ");
        temp = front;
    }
}

```

```
while(temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
}
return 0;
}
```

Output

```
Enter number of elements in queue: 6
12 43 55 67 78 98
Queue elements are: 12 43 55 67 78 98

=== Code Execution Successful ===
```